You can order print and ebook versions of *Think Bayes 2e* from Bookshop.org and Amazon.

Odds and Addends

```
# install empiricaldist if necessary

try:
    import empiricaldist
except ImportError:
    !pip install empiricaldist
    import empiricaldist
```

```
# Get utils.py

from os.path import basename, exists

def download(url):
    filename = basename(url)
    if not exists(filename):
        from urllib.request import urlretrieve
        local, _ = urlretrieve(url, filename)
        print('Downloaded ' + local)

download('https://github.com/AllenDowney/ThinkBayes2/raw/master/soln/utils.py')
```

```
from utils import set_pyplot_params
set_pyplot_params()
```

This chapter presents a new way to represent a degree of certainty, **odds**, and a new form of Bayes's Theorem, called **Bayes's Rule**. Bayes's Rule is convenient if you want to do a Bayesian update on paper or in your head. It also sheds light on the important idea of **evidence** and how we can quantify the strength of evidence.

The second part of the chapter is about "addends", that is, quantities being added, and how we can compute their distributions. We'll define functions that compute the distribution of sums, differences, products, and other operations. Then we'll use those distributions as part of a Bayesian update.

Odds

One way to represent a probability is with a number between 0 and 1, but that's not the only way. If you have ever bet on a football game or a horse race, you have probably encountered another representation of probability, called **odds**.

You might have heard expressions like "the odds are three to one", but you might not know what that means. The **odds in favor** of an event are the ratio of the probability it will occur to the probability that it will not.

The following function does this calculation.

```
def odds(p):
    return p / (1-p)
```

For example, if my team has a 75% chance of winning, the odds in their favor are three to one, because the chance of winning is three times the chance of losing.

```
odds(0.75)
```

```
3.0
```

You can write odds in decimal form, but it is also common to write them as a ratio of integers. So "three to one" is sometimes written 3 : 1.

When probabilities are low, it is more common to report the **odds against** rather than the odds in favor. For example, if my horse has a 10% chance of winning, the odds in favor are 1 : 9.

```
odds(0.1)
```

```
0.11111111111111112
```

But in that case it would be more common I to say that the odds against are 9 : 1.

```
odds(0.9)
```

```
9.000000000000002
```

Given the odds in favor, in decimal form, you can convert to probability like this:

```
def prob(o):
    return o / (o+1)
```

For example, if the odds are $3/2$, the corresponding probability is $3/5$:

```
prob(3/2)
```

```
0.6
```

Or if you represent odds with a numerator and denominator, you can convert to probability like this:

```
def prob2(yes, no):
    return yes / (yes + no)
```

```
prob2(3, 2)
```

```
0.6
```

Probabilities and odds are different representations of the same information; given either one, you can compute the other. But some computations are easier when we work with odds, as we'll see in the next section, and some computations are even easier with log odds, which we'll see later.

Bayes's Rule

So far we have worked with Bayes's theorem in the "probability form":

$$P(H|D) = \frac{P(H)\ P(D|H)}{P(D)}$$

Writing $\text{odds}(A)$ for odds in favor of A, we can express Bayes's Theorem in "odds form":

$$\text{odds}(A|D) = \text{odds}(A)\ \frac{P(D|A)}{P(D|B)}$$

This is Bayes's Rule, which says that the posterior odds are the prior odds times the likelihood ratio. Bayes's Rule is convenient for computing a Bayesian update on paper or in your head. For example, let's go back to the cookie problem:

> *Suppose there are two bowls of cookies. Bowl 1 contains 30 vanilla cookies and 10 chocolate cookies. Bowl 2 contains 20 of each. Now suppose you choose one of the bowls at random and, without looking, select a cookie at random. The cookie is vanilla. What is the probability that it came from Bowl 1?*

The prior probability is 50%, so the prior odds are 1. The likelihood ratio is $\frac{3}{4}/\frac{1}{2}$, or $3/2$. So the posterior odds are $3/2$, which corresponds to probability $3/5$.

```
prior_odds = 1
likelihood_ratio = (3/4) / (1/2)
post_odds = prior_odds * likelihood_ratio
post_odds
```

```
1.5
```

```
post_prob = prob(post_odds)
post_prob
```

```
0.6
```

If we draw another cookie and it's chocolate, we can do another update:

```
likelihood_ratio = (1/4) / (1/2)
post_odds *= likelihood_ratio
post_odds
```

```
0.75
```

And convert back to probability.

```
post_prob = prob(post_odds)
post_prob
```

```
0.42857142857142855
```

Oliver's Blood

I'll use Bayes's Rule to solve another problem from MacKay's *Information Theory, Inference, and Learning Algorithms*:

> *Two people have left traces of their own blood at the scene of a crime. A suspect, Oliver, is tested and found to have type 'O' blood. The blood groups of the two traces are found to be of type 'O' (a common type in the local population, having frequency 60%) and of type 'AB' (a rare type, with frequency 1%). Do these data [the traces found at the scene] give evidence in favor of the proposition that Oliver was one of the people [who left blood at the scene]?*

To answer this question, we need to think about what it means for data to give evidence in favor of (or against) a hypothesis. Intuitively, we might say that data favor a hypothesis if the hypothesis is more likely in light of the data than it was before.

In the cookie problem, the prior odds are 1, which corresponds to probability 50%. The posterior odds are $3/2$, or probability 60%. So the vanilla cookie is evidence in favor of Bowl 1.

Bayes's Rule provides a way to make this intuition more precise. Again

$$\text{odds}(A|D) = \text{odds}(A)\,\frac{P(D|A)}{P(D|B)}$$

Dividing through by $\text{odds}(A)$, we get:

$$\frac{\text{odds}(A|D)}{\text{odds}(A)} = \frac{P(D|A)}{P(D|B)}$$

The term on the left is the ratio of the posterior and prior odds. The term on the right is the likelihood ratio, also called the **Bayes factor**.

If the Bayes factor is greater than 1, that means that the data were more likely under A than under B. And that means that the odds are greater, in light of the data, than they were before.

If the Bayes factor is less than 1, that means the data were less likely under A than under B, so the odds in favor of A go down.

Finally, if the Bayes factor is exactly 1, the data are equally likely under either hypothesis, so the odds do not change.

Let's apply that to the problem at hand. If Oliver is one of the people who left blood at the crime scene, he accounts for the 'O' sample; in that case, the probability of the data is the probability that a random member of the population has type 'AB' blood, which is 1%.

If Oliver did not leave blood at the scene, we have two samples to account for. If we choose two random people from the population, what is the chance of finding one with type 'O' and one with type 'AB'? Well, there are two ways it might happen:

- The first person might have 'O' and the second 'AB',
- Or the first person might have 'AB' and the second 'O'.

The probability of either combination is $(0.6)(0.01)$, which is 0.6%, so the total probability is twice that, or 1.2%. So the data are a little more likely if Oliver is *not* one of the people who left blood at the scene.

We can use these probabilities to compute the likelihood ratio:

```
like1 = 0.01
like2 = 2 * 0.6 * 0.01

likelihood_ratio = like1 / like2
likelihood_ratio
```

```
0.8333333333333334
```

Since the likelihood ratio is less than 1, the blood tests are evidence *against* the hypothesis that Oliver left blood at the scence.

But it is weak evidence. For example, if the prior odds were 1 (that is, 50% probability), the posterior odds would be 0.83, which corresponds to a probability of 45%:

```
post_odds = 1 * like1 / like2
prob(post_odds)
```

```
0.45454545454545453
```

So this evidence doesn't "move the needle" very much.

This example is a little contrived, but it demonstrates the counterintuitive result that data *consistent* with a hypothesis are not necessarily *in favor of* the hypothesis.

If this result still bothers you, this way of thinking might help: the data consist of a common event, type 'O' blood, and a rare event, type 'AB' blood. If Oliver accounts for the common event, that leaves the rare event unexplained. If Oliver doesn't account for the 'O' blood, we have two chances to find someone in the population with 'AB' blood. And that factor of two makes the difference.

Exercise: Suppose that based on other evidence, you prior belief in Oliver's guilt is 90%. How much would the blood evidence in this section change your beliefs? What if you initially thought there was only a 10% chance of his guilt?

```
# Solution goes here
```

```
# Solution goes here
```

Addends

The second half of this chapter is about distributions of sums and results of other operations. We'll start with a forward problem, where we are given the inputs and compute the distribution of the output. Then we'll work on inverse problems, where we are given the outputs and we compute the distribution of the inputs.

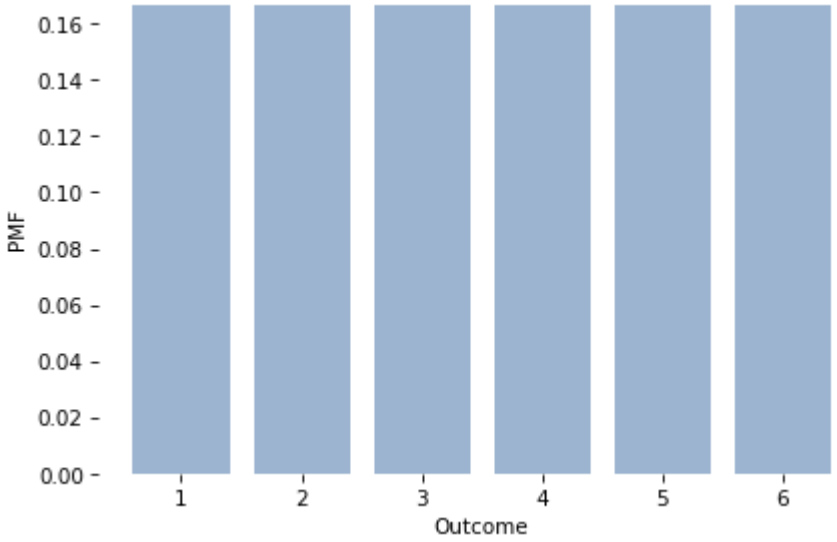
As a first example, suppose you roll two dice and add them up. What is the distribution of the sum? I'll use the following function to create a `Pmf` that represents the possible outcomes of a die:

```
import numpy as np
from empiricaldist import Pmf

def make_die(sides):
    outcomes = np.arange(1, sides+1)
    die = Pmf(1/sides, outcomes)
    return die
```

On a six-sided die, the outcomes are 1 through 6, all equally likely.

```
die = make_die(6)
```

```
from utils import decorate

die.bar(alpha=0.4)
decorate(xlabel='Outcome',
         ylabel='PMF')
```



If we roll two dice and add them up, there are 11 possible outcomes, 2 through 12, but they are not equally likely. To compute the distribution of the sum, we have to enumerate the possible outcomes.

And that's how this function works:

```
def add_dist(pmf1, pmf2):
    """Compute the distribution of a sum."""
    res = Pmf()
    for q1, p1 in pmf1.items():
        for q2, p2 in pmf2.items():
            q = q1 + q2
            p = p1 * p2
            res[q] = res(q) + p
    return res
```

The parameters are `Pmf` objects representing distributions.

The loops iterate though the quantities and probabilities in the `Pmf` objects. Each time through the loop `q` gets the sum of a pair of quantities, and `p` gets the probability of the pair. Because the same sum might appear more than once, we have to add up the total probability for each sum.

Notice a subtle element of this line:

```
res[q] = res(q) + p
```

I use parentheses on the right side of the assignment, which returns 0 if `q` does not appear yet in `res`. I use brackets on the left side of the assignment to create or update an element in `res`; using parentheses on the left side would not work.

`Pmf` provides `add_dist`, which does the same thing. You can call it as a method, like this:

```
twice = die.add_dist(die)
```

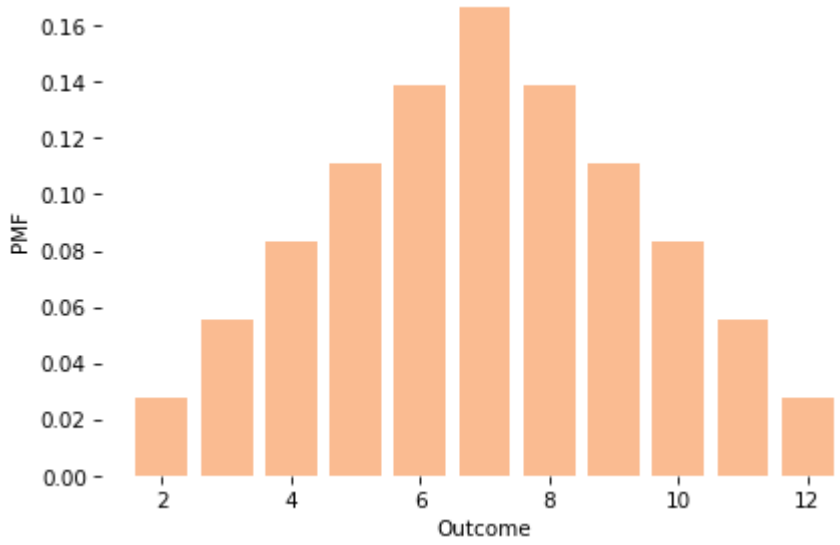
Or as a function, like this:

```
twice = Pmf.add_dist(die, die)
```

And here's what the result looks like:

```
from utils import decorate

def decorate_dice(title=''):
    decorate(xlabel='Outcome',
             ylabel='PMF',
             title=title)
```

```
twice = add_dist(die, die)
twice.bar(color='C1', alpha=0.5)
decorate_dice()
```



If we have a sequence of `Pmf` objects that represent dice, we can compute the distribution of the sum like this:

```
def add_dist_seq(seq):
    """Compute Pmf of the sum of values from seq."""
    total = seq[0]
    for other in seq[1:]:
        total = total.add_dist(other)
    return total
```

As an example, we can make a list of three dice like this:

```
dice = [die] * 3
```

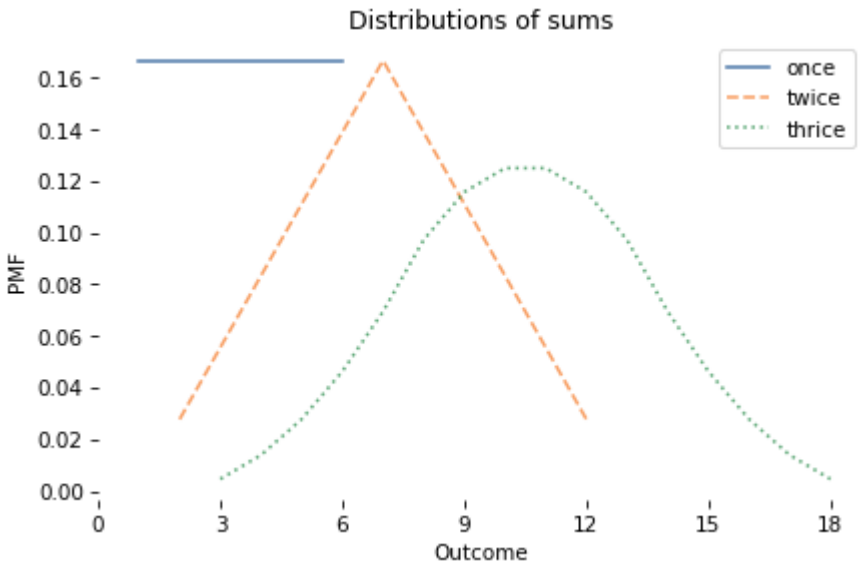
And we can compute the distribution of their sum like this.

```
thrice = add_dist_seq(dice)
```

The following figure shows what these three distributions look like:

- The distribution of a single die is uniform from 1 to 6.
- The sum of two dice has a triangle distribution between 2 and 12.
- The sum of three dice has a bell-shaped distribution between 3 and 18.

```
import matplotlib.pyplot as plt

die.plot(label='once')
twice.plot(label='twice', ls='--')
thrice.plot(label='thrice', ls=':')

plt.xticks([0,3,6,9,12,15,18])
decorate_dice(title='Distributions of sums')
```



As an aside, this example demonstrates the Central Limit Theorem, which says that the distribution of a sum converges on a bell-shaped normal distribution, at least under some conditions.

Gluten Sensitivity

In 2015 I read a paper that tested whether people diagnosed with gluten sensitivity (but not celiac disease) were able to distinguish gluten flour from non-gluten flour in a blind challenge (you can read the paper here).

Out of 35 subjects, 12 correctly identified the gluten flour based on resumption of symptoms while they were eating it. Another 17 wrongly identified the gluten-free flour based on their symptoms, and 6 were unable to distinguish.

The authors conclude, "Double-blind gluten challenge induces symptom recurrence in just one-third of patients."

This conclusion seems odd to me, because if none of the patients were sensitive to gluten, we would expect some of them to identify the gluten flour by chance. So here's the question: based on this data, how many of the subjects are sensitive to gluten and how many are guessing?

We can use Bayes's Theorem to answer this question, but first we have to make some modeling decisions. I'll assume:

- People who are sensitive to gluten have a 95% chance of correctly identifying gluten flour under the challenge conditions, and
- People who are not sensitive have a 40% chance of identifying the gluten flour by chance (and a 60% chance of either choosing the other flour or failing to distinguish).

These particular values are arbitrary, but the results are not sensitive to these choices.

I will solve this problem in two steps. First, assuming that we know how many subjects are sensitive, I will compute the distribution of the data. Then, using the likelihood of the data, I will compute the posterior distribution of the number of sensitive patients.

The first is the **forward problem**; the second is the **inverse problem**.

The Forward Problem

Suppose we know that 10 of the 35 subjects are sensitive to gluten. That means that 25 are not:

```
n = 35
num_sensitive = 10
num_insensitive = n - num_sensitive
```

Each sensitive subject has a 95% chance of identifying the gluten flour, so the number of correct identifications follows a binomial distribution.

I'll use `make_binomial`, which we defined in <<_TheBinomialDistribution>>, to make a `Pmf` that represents the binomial distribution.

```
from utils import make_binomial

dist_sensitive = make_binomial(num_sensitive, 0.95)
dist_insensitive = make_binomial(num_insensitive, 0.40)
```

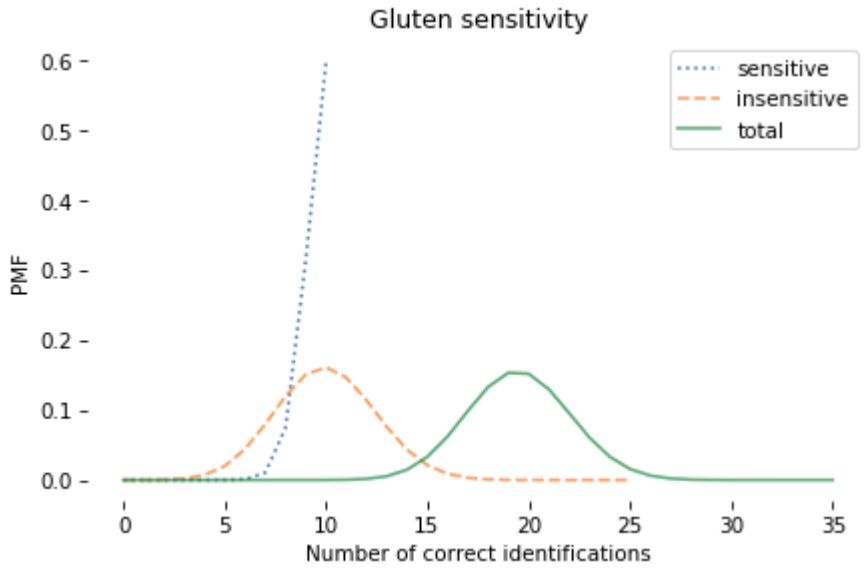
The results are the distributions for the number of correct identifications in each group.

Now we can use `add_dist` to compute the distribution of the total number of correct identifications:

```
dist_total = Pmf.add_dist(dist_sensitive, dist_insensitive)
```

Here are the results:

```
dist_sensitive.plot(label='sensitive', ls=':')
dist_insensitive.plot(label='insensitive', ls='--')
dist_total.plot(label='total')

decorate(xlabel='Number of correct identifications',
         ylabel='PMF',
         title='Gluten sensitivity')
```



We expect most of the sensitive subjects to identify the gluten flour correctly. Of the 25 insensitive subjects, we expect about 10 to identify the gluten flour by chance. So we expect about 20 correct identifications in total.

This is the answer to the forward problem: given the number of sensitive subjects, we can compute the distribution of the data.

The Inverse Problem

Now let's solve the inverse problem: given the data, we'll compute the posterior distribution of the number of sensitive subjects.

Here's how. I'll loop through the possible values of `num_sensitive` and compute the distribution of the data for each:

```
import pandas as pd

table = pd.DataFrame()
for num_sensitive in range(0, n+1):
    num_insensitive = n - num_sensitive
    dist_sensitive = make_binomial(num_sensitive, 0.95)
    dist_insensitive = make_binomial(num_insensitive, 0.4)
    dist_total = Pmf.add_dist(dist_sensitive, dist_insensitive)
    table[num_sensitive] = dist_total
```

The loop enumerates the possible values of `num_sensitive`. For each value, it computes the distribution of the total number of correct identifications, and stores the result as a column in a Pandas `DataFrame`.

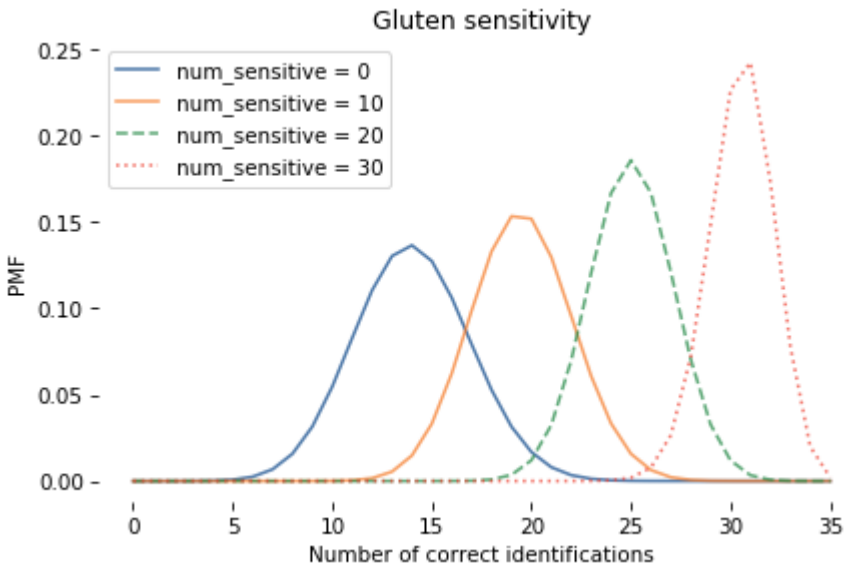
```
table.head(3)
```

	0	1	2	3	4	
0	1.719071e-08	1.432559e-09	1.193799e-10	9.948326e-12	8.290272e-13	6.90856(
1	4.011165e-07	5.968996e-08	7.162795e-09	7.792856e-10	8.013930e-11	7.94484
2	4.545987e-06	9.741401e-07	1.709122e-07	2.506426e-08	3.269131e-09	3.94018

3 rows × 36 columns

The following figure shows selected columns from the `DataFrame`, corresponding to different hypothetical values of `num_sensitive`:

```
table[0].plot(label='num_sensitive = 0')
table[10].plot(label='num_sensitive = 10')
table[20].plot(label='num_sensitive = 20', ls='--')
table[30].plot(label='num_sensitive = 30', ls=':')

decorate(xlabel='Number of correct identifications',
         ylabel='PMF',
         title='Gluten sensitivity')
```



Now we can use this table to compute the likelihood of the data:

```
likelihood1 = table.loc[12]
```

`loc` selects a row from the `DataFrame`. The row with index 12 contains the probability of 12 correct identifications for each hypothetical value of `num_sensitive`. And that's exactly the likelihood we need to do a Bayesian update.

I'll use a uniform prior, which implies that I would be equally surprised by any value of `num_sensitive`:

```
hypos = np.arange(n+1)
prior = Pmf(1, hypos)
```

And here's the update:

```
posterior1 = prior * likelihood1
posterior1.normalize()
```

```
np.float64(0.4754741648615128)
```

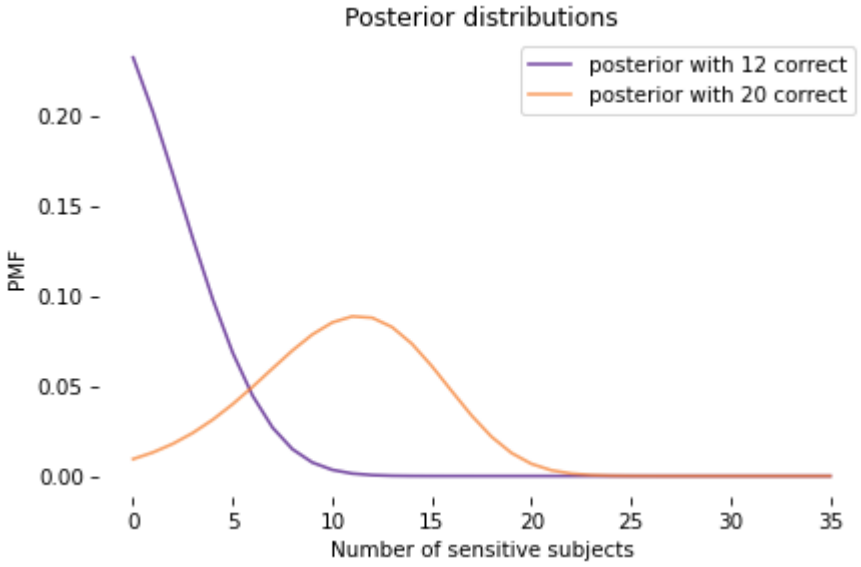
For comparison, I also compute the posterior for another possible outcome, 20 correct identifications.

```
likelihood2 = table.loc[20]
posterior2 = prior * likelihood2
posterior2.normalize()
```

```
np.float64(1.7818649765887382)
```

The following figure shows posterior distributions of `num_sensitive` based on the actual data, 12 correct identifications, and the other possible outcome, 20 correct identifications.

```
posterior1.plot(label='posterior with 12 correct', color='C4')
posterior2.plot(label='posterior with 20 correct', color='C1')

decorate(xlabel='Number of sensitive subjects',
         ylabel='PMF',
         title='Posterior distributions')
```



With 12 correct identifications, the most likely conclusion is that none of the subjects are sensitive to gluten. If there had been 20 correct identifications, the most likely conclusion would be that 11-12 of the subjects were sensitive.

```
posterior1.max_prob()
```

```
np.int64(0)
```

```
posterior2.max_prob()
```

```
np.int64(11)
```

Summary

This chapter presents two topics that are almost unrelated except that they make the title of the chapter catchy.

The first part of the chapter is about Bayes's Rule, evidence, and how we can quantify the strength of evidence using a likelihood ratio or Bayes factor.

The second part is about `add_dist`, which computes the distribution of a sum. We can use this function to solve forward and inverse problems; that is, given the parameters of a system, we can compute the distribution of the data or, given the data, we can compute the distribution of the parameters.

In the next chapter, we'll compute distributions for minimums and maximums, and use them to solve more Bayesian problems. But first you might want to work on these exercises.

Exercises

Exercise: Let's use Bayes's Rule to solve the Elvis problem from <<_Distributions>>:

> *Elvis Presley had a twin brother who died at birth. What is the probability that Elvis was an identical twin?*

In 1935, about 2/3 of twins were fraternal and 1/3 were identical. The question contains two pieces of information we can use to update this prior.

- First, Elvis's twin was also male, which is more likely if they were identical twins, with a likelihood ratio of 2.
- Also, Elvis's twin died at birth, which is more likely if they were identical twins, with a likelihood ratio of 1.25.

If you are curious about where those numbers come from, I wrote a blog post about it.

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

Exercise: The following is an interview question that appeared on glassdoor.com, attributed to Facebook:

> *You're about to get on a plane to Seattle. You want to know if you should bring an umbrella. You call 3 random friends of yours who live there and ask each independently if it's raining. Each of your friends has a 2/3 chance of telling you the truth and a 1/3 chance of messing with you by lying. All 3 friends tell you that "Yes" it is raining. What is the probability that it's actually raining in Seattle?*

Use Bayes's Rule to solve this problem. As a prior you can assume that it rains in Seattle about 10% of the time.

This question causes some confusion about the differences between Bayesian and frequentist interpretations of probability; if you are curious about this point, I wrote a blog article about it.

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

Exercise: According to the CDC, people who smoke are about 25 times more likely to develop lung cancer than nonsmokers.

Also according to the CDC, about 14% of adults in the U.S. are smokers. If you learn that someone has lung cancer, what is the probability they are a smoker?

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

Exercise: In *Dungeons & Dragons*, the amount of damage a goblin can withstand is the sum of two six-sided dice. The amount of damage you inflict with a short sword is determined by rolling one six-sided die. A goblin is defeated if the total damage you inflict is greater than or equal to the amount it can withstand.

Suppose you are fighting a goblin and you have already inflicted 3 points of damage. What is your probability of defeating the goblin with your next successful attack?

Hint: You can use `Pmf.sub_dist` to subtract a constant amount, like 3, from a `Pmf`.

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

Exercise: Suppose I have a box with a 6-sided die, an 8-sided die, and a 12-sided die. I choose one of the dice at random, roll it twice, multiply the outcomes, and report that the product is 12. What is the probability that I chose the 8-sided die?

Hint: `Pmf` provides a function called `mul_dist` that takes two `Pmf` objects and returns a `Pmf` that represents the distribution of the product.

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

Exercise: *Betrayal at House on the Hill* is a strategy game in which characters with different attributes explore a haunted house. Depending on their attributes, the characters roll different numbers of dice. For example, if attempting a task that depends on knowledge, Professor Longfellow rolls 5 dice, Madame Zostra rolls 4, and Ox Bellows rolls 3. Each die yields 0, 1, or 2 with equal probability.

If a randomly chosen character attempts a task three times and rolls a total of 3 on the first attempt, 4 on the second, and 5 on the third, which character do you think it was?

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

Exercise: There are 538 members of the United States Congress. Suppose we audit their investment portfolios and find that 312 of them out-perform the market. Let's assume that an honest member of Congress has only a 50% chance of out-performing the market, but a dishonest member who trades on inside information has a 90% chance. How many members of Congress are honest?

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

Think Bayes, Second Edition